UCAM

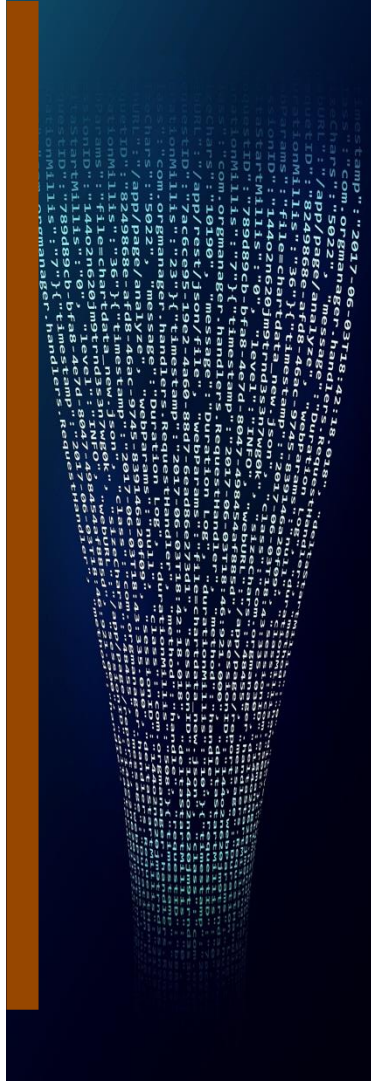
Tema 3

Entrada/Salida Formateada

Fundamentos de Programación I

Juan Bonastre Egea / Angélica Guzmán Ponce

Grado en Ingeniería Informática



ÍNDICE

CONTENIDOS

1. La función `printf`

2. La función `scanf`



Imagen 1

Teclado de un ordenador

1. La función `printf`

Nota. Adaptado de *Cerrar foto de teclado*, por Hitarth Jadhav, 2016, Pexels.

1. printf / scanf

- ✓ Las funciones **printf** / **scanf** son dos de las funciones más utilizadas en C ya que proporcionan entrada/salida de información en nuestro programa.
- ✓ Concretamente proporcionan E/S formateada. De ahí la **f** final de **printf** / **scanf**.
- ✓ Más adelante veremos la E/S de C con mucho más en detalle.



1. La función `printf` (i)

- ✓ La función **`printf`** está diseñada para mostrar el contenido de una cadena.
  - A la cadena proporcionada en la función **`printf`** se la denomina **cadena de formato** (*format string*).
  - En la cadena de formato se pueden **insertar ciertos puntos** que se **sustituirán por valores concretos** también proporcionados en la llamada.
- ✓ El prototipo de la función es el siguiente:

```
int printf(const char *format, expr1, expr2,...);
```

1. La función `printf` (ii)

```
int printf(const char *format, expr1, expr2, ...);
```

- ✓ La cadena de formato puede contener caracteres ordinarios y **especificaciones de conversión**.
  - Las especificaciones de conversión comienzan por %
  - Una especificación de conversión indica un lugar a ser 'rellenado' cuando se muestra la cadena.



1. La función `printf` (iii)

- El carácter tras `%` indica cómo realizar la conversión.
  - Ej.: la conversión `%d` le indica a **`printf`** que hay que convertir el valor suministrado e imprimir un número entero.
  - Ej.: la conversión `%f` le indica a **`printf`** que hay que convertir el valor suministrado e imprimir un número real.

✓ Los caracteres ordinarios se muestran tal cual aparecen.



1. La función `printf` (iv)

Tabla de formatos

Especificador de formato	Salida	Ejemplo
%d o %i	Entero con signo	392
%u	Entero sin signo	7235
%f / %F	Float	392.65
%e / %E	Notación científica	3.926e+02
%g / %G	Representación más corta (%e o %f)	392.65
%c	Carácter	‘A’
%s	Cadena de caracteres	“Sample”

Tabla obtenida de https://cplusplus.com/reference/cstdio/printf/

1. La función `printf` (v)

Sub-especificadores de longitud

✓ Los **sub-especificadores de longitud** modifican la longitud del tipo de dato. Por ejemplo:

- %hd: short int
- %hu: unsigned short int
- %ld: long int
- %Lf: long double

length	d i	u	f F e E g G	c	s
(none)	int	unsigned int	double	int	char*
hh	signed char	unsigned char			
h	short int	unsigned short int			
l	long int	unsigned long int		wint_t	wchar_t*
ll	long long int	unsigned long long int			
L			long double		

Tabla obtenida de https://cplusplus.com/reference/cstdio/printf/

1. Ejemplo de uso de la función `printf` (vi)

```
int n1, n2;

n1 = 24;

n2 = 2;

printf("%d x %d = %d", n1, n2, n1 * n2 );
```

... mostrará la cadena:

24 x 2 = 48

(el carácter '**x**' y el signo '**=**' se copian tal cual)

1. Sobre los parámetros de `printf`

- ✓ **`printf` NO comprueba** que el número de conversiones coincida con el número de parámetros pasados.
  - Esto puede plantear numerosos problemas si no se utiliza **`printf` con** cuidado.
- ✓ Ejemplos:

```
printf ("%d %d \n",x,y,z);     // MAL

printf ("%d %d \n",a);         // PEOR

printf ("%f %d \n",i,f);       // MAL (i tipo int, f tipo float)
```

1. Especificaciones de conversión (i)

- ✓ Las **especificaciones de conversión** proporcionan un gran control sobre la apariencia de la salida.
- ✓ Pero, al mismo tiempo, son complicadas y difíciles de leer.
- ✓ Una especificación de conversión tiene el siguiente formato:

```
%[flags][ancho_mínimo][.precision]especificador
```

  - *ancho mínimo* ➔ mínimo número de caracteres a mostrar.
  - *precisión* ➔ su significado depende de la conversión a realizar. En el caso de **`float`** (**`%f`**) especifica el número de dígitos decimales.

1. Especificaciones de conversión (ii)

✓ Las conversiones de números más comunes son:

`%d` Muestra un **número entero** en **base 10**

La precisión en este caso indica el mínimo número de dígitos a mostrar (%d equivale a %.1d).

`%e` Muestra un **número en coma flotante** en **formato exponencial** (notación científica).

La precisión indica el número de dígitos después del punto decimal.



1. Especificaciones de conversión (iii)

`%f` Muestra un **número en coma flotante** en **formato fijo decimal** sin exponente.

La precisión tiene el mismo significado que para %e.

`%g` Muestra un **número en coma flotante** en **formato exponencial o decimal fijo** dependiendo del tamaño del número.

La precisión indica el número de dígitos significativos a visualizar.

1. Especificaciones de conversión (iv)

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
     float f;
     int n;
     n = 20;
     f = 172.346;
     printf("Conversiones de números enteros:\n");
     printf("%d|%5d|%-5d|%5.3d|\n", n, n, n, n);
     printf("Conversiones de números en coma flotante:\n");
     printf("%10.3f|%10.4f|%10.2f|%10.3e|%-10g|\n", f, f, f, f, f);

     return 0;
}
```

1. Especificaciones de conversión (v)

- ✓ La salida del programa anterior:
- ✓ Conversiones de números enteros:

```
20|   20|20   |  020|
```

- ✓ Conversiones de números en coma flotante:

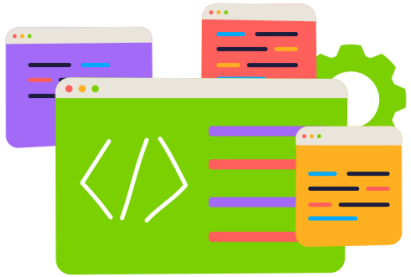
```
172.346|  172.3460|    172.35| 1.723e+02|172.346   |
```

Nota. El estándar C exige al menos dos dígitos en el exponente, por lo que *%10.3e* imprime *1.723e+02* en macOS y Linux. Algunas versiones de MinGW/Code::Blocks en Windows enlazan con el runtime antiguo de Microsoft y muestran tres: *1.723e+002*. Es el mismo número.

1. Especificaciones de conversión (vi)

- ✓ Para mostrar un porcentaje...

```
printf("%d %%", a);
```



1. Secuencias de escape (i)

✓ Una **secuencia de escape** permite que las cadenas de caracteres (**strings**) contengan **caracteres que podrían causar problemas** al compilador, incluyendo:

- Caracteres no imprimibles (de control).
- Caracteres que tienen un significado especial para el compilador (como “ ó \).

✓ Ejemplos:

\a	Alerta (bell).
\b	Retroceso.
\n	Nueva línea.
\t	Tabulador.

1. Secuencias de escape (ii)

- ✓ Las secuencias de escape representan acciones a realizar sobre la impresión.
- ✓ El carácter de escape \ también se utiliza para poder mostrar caracteres como \ ó "
  - **\r** realiza un retorno de carro.
  - **** muestra una contrabarra
- ✓ Ejemplos:

```
printf("Y él dijo: \" Me voy !!\"\n");
```



Imagen 2

Ordenador portátil

2. La función `scanf`

Nota. Adaptado de *Ordenador portátil gris*, por JÉSHOOT, 2017, Pexels.

2. La función `scanf` (i)

- ✓ **`scanf`** es a la entrada de datos lo que **`printf`** es a la salida.
- ✓ **`scanf`** es capaz de leer datos de entrada según se especifique en una cadena de formato.
- ✓ Las **especificaciones de conversión** que se pueden especificar son idénticas a las de **`printf`**.



2. La función `scanf`: ejemplo (ii)

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
      int x, y;
      float a;
      scanf("%d%d%f", &x, &y, &a);
      printf("Los números leídos son:\n");
      printf("x = %d\n", x);
      printf("y = %d\n", y);
      printf("a = %f\n", a);
      return 0;
}
```

2. La función `scanf` (iii)

- ✓ Como en **`printf`**, es labor del programador comprobar que **el número de conversiones a realizar debe coincidir con el número de parámetros pasados**.
- ✓ Con **`scanf`**, normalmente, cada variable debe ir precedida del carácter **`&`**.
  - Olvidarlo puede tener resultados impredecibles.
  - El compilador puede dar un aviso (*warning*).
- ✓ **`scanf`** es potente, pero poco utilizada. Es mejor leer una cadena de caracteres y realizar más tarde la conversión.
  - Solo la utilizaremos por ahora.

2. ¿Cómo funciona `scanf`? (i)

- ✓ Realmente **`scanf`** intenta hacer coincidir la entrada con la cadena de conversión (*pattern matching*).
  - Va leyendo la entrada carácter a carácter, intentando hacerla coincidir con la cadena de conversión. Cuando encuentra una especificación de conversión intenta hacerla coincidir con los datos de entrada.
  - **`scanf` se detiene** si no encuentra caracteres que coincidan con los indicados en la cadena de conversión.



¡Una nueva llamada a `scanf` seguirá leyendo desde donde se quedó el anterior `scanf`!

2. ¿Cómo funciona `scanf`? (ii)

- ✓ Si **`scanf`** está buscando el comienzo de un número, **ignorará los espacios en blanco, incluyendo tabuladores y saltos de línea.**
- ✓ La siguiente llamada a **`scanf`**:

```
scanf("%d%d%f%f", &i, &j, &x, &y);
```

... con la siguiente entrada de datos

```
1 -20    .3    -4.0e3
```

... es vista por **`scanf`** como un flujo continuo de caracteres en la que se leen los 4 números:

```
1NL···-20·NL···.3NL····-4.0e3NL
```

2. ¿Cómo funciona `scanf`? (iii)

- ✓ Cuando **`scanf`** tiene que **leer un entero** busca:
  - Signo más o menos (opcional).
  - Un conjunto de dígitos hasta que aparezca un carácter que no lo sea.
- ✓ Cuando **`scanf`** tiene que **leer un número real** busca:
  - Un signo más o menos (opcional).
  - Un conjunto de dígitos (con un posible punto decimal), seguido por…
    - Un exponente (opcional), que consiste en un carácter e (ó E), un signo opcional y uno o más dígitos.
  - Las conversiones **`%e`**, **`%f`** y **`%g`** **son intercambiables** con **`scanf`**.

2. ¿Cómo funciona `scanf`? (iv)

- ✓ Cuando **`scanf`** encuentra un carácter que no forma parte de la actual conversión, **`scanf`**
  - Lo devuelve a la entrada para la siguiente conversión.
  - También podría ser empleado **en la siguiente llamada a `scanf` .**



2. Caracteres ordinarios en `scanf` (i)

- ✓ La cadena de conversión de **`scanf`** **puede contener caracteres ordinarios**. Estos caracteres tienen un tratamiento especial según sean:
  - **Caracteres en blanco**: **`scanf`** lee cualquier conjunto de caracteres en blanco (o ninguno).
  - **Otros caracteres**: **scanf** intentará leer de la entrada el carácter.
    - Si lo encuentra seguirá procesando la cadena de conversión
    - En caso contrario, **`scanf`** termina.

2. Caracteres ordinarios en `scanf` (ii)

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{     int dia;
      int mes;
      int agno;
      printf("Introduzca una fecha con el formato dd/mm/yyyy: ");
      scanf("%d / %d / %d", &dia, &mes, &agno);
      printf("La fecha es:\n\t");
      printf("%d/%d/%d\n", dia, mes, agno);
      return 0;
}
```

2. Caracteres ordinarios en `scanf` (iii)

```
Introduzca una fecha con el formato dd/mm/yyyy: 19/03/2009
La fecha es:        19/3/2009
**********************************************************************
Introduzca una fecha con el formato dd/mm/yyyy:  19   /   3   /   2008
La fecha es:        19/3/2008
**********************************************************************
Introduzca una fecha con el formato dd/mm/yyyy: 1*1*2010
La fecha es:        1/134513737/-1078788024
**********************************************************************
Introduzca una fecha con el formato dd/mm/yyyy: pepe
La fecha es:        -1208447568/134513737/-1082114792
**********************************************************************
Introduzca una fecha con el formato dd/mm/yyyy: 1/enero/2010
La fecha es:        1/134513737/-1077333592
```

Práctica en el aula
printf y scanf a fondo

1. Muestra con una sola llamada a **`printf`**: Me llamo "Ana" y la ruta "C:\FUP1\practicas”
2. Lee un número real y muéstralo con 1, 3 y 6 decimales (**`%.1f`**, **`%.3f`**, **`%.6f`**).
3. Lee día, mes y año y muestra la fecha como dd/mm/aaaa usando **`%02d`** y **`%04d`**.
4. Lee 3 precios y muéstralos alineados a la derecha con **`%10.2f`**, uno por línea.
5. Lee una hora hh:mm:ss con **`scanf("%d:%d:%d", ...)`** y calcula los segundos transcurridos desde las 00:00:00.

REFERENCIAS

BIBLIOGRÁFICAS

1. King K.N. C Programming. A modern approach. 2ªed. Berkley: W.W. Norton & Company; 2008. Chapter 3.
2. Joyanes Aguilar L. Fundamentos de programación, algoritmos, estructura de datos y objetos, 5ª edición, McGraw-Hill Interamericana de España S.L.; 2020. ISBN: 9781456279448

Angélica Guzmán Ponce

aguzmán@ucam.edu

UCAM Universidad Católica de Murcia



© UCAM